DIP V2 & V3 Communication Protocol

RS485 Control

Adapt to MODBUS communication protocol

The protocol adopts RTU MODBUS protocol, and Modbus knowledge reference: https://en.wikipedia.org/wiki/Modbus#Modbus_RTU_frame_format_(primarily_used_on_asynchronous_serial_data_lines_like_RS-485/EIA-485)

The specific agreement content is as follows:

Coil

Address	Parameter	Remarks
0x0001	Motor start stop control: 0 stop, 1 start	
0x0002	Motor direction: 0 counterclockwise, 1 clockwise	
0x0003	Flash save: Write 1 save, automatic reset	
0x0004	Operation mode: 0 is time mode, 1 is volume mode	Default is time mode

Save Register

Address	Parameter	Remarks
0x4001~0x4002	Motor speed (rpm)	Floating point number
0x4003~0x4004	Maximum motor speed (rpm)	Floating point number
0x4005~0x4006	Running time H (ms) or running volume ml	Run continuously for a duration of 0; Time mode data format: 32-bit unsigned integer, small end mode byte exchange (CDAB); Volume mode data format: 32-bit single precision floating-point, small end mode byte exchange (CDAB);
0x4007~0x4008	Run real-time duration H (ms) or run real-time volume ml	After running the new configuration, reset to zero by yourself, and reset to zero after selecting a new unit; Time mode data format: 32-bit unsigned integer, small end mode byte exchange (CDAB); Volume mode data format: 32-bit

		single precision floating-point, small end mode byte exchange (CDAB);
0x4009~0x400a	Accumulated running time H (ms) or accumulated running volume ml	If it is time, the maximum duration can be saved for 49 days, reset to zero after selecting a new unit; Time mode data format: 32-bit unsigned integer, small end mode byte exchange (CDAB); Volume mode data format: 32-bit single precision floating-point, small end mode byte exchange (CDAB);
0x400b~0x400c	Flow rate (ml/min)/Speed (rpm)	Floating point number, the same calibration coefficient used for calibration in rotary encoder mode, can also be written by users themselves
0x400d	Foot switch input mode, same as rotary encoder setting option	
0x400e	Liquid level input mode, same as rotary encoder setting option	
0x400f	State output mode, same as setting options for rotary encoder	
0x4010	0~31	Communication address
0x4011	0:1200 1:2400 2:4800 3:9600 4:19200 5:38400 6:57600 7:115200	Baud rate indexset

The motor speed is a floating point number. During Modbus data transmission, it is necessary to first convert the floating point number to hexadecimal data. The conversion idea is that the floating point number is stored in bytes in memory, and we can obtain the corresponding bytes of the floating point number.

Here are two examples of Modbus, one of which is an example of setting the speed floating point number:

Set the speed to 50.0rpm

Command: 01 10 40 01 00 02 04 00 00 42 48 33 36

Command note:

01: Modbus address

10: Modbus function code

40 01: Modbus start address

00 02: Number of registers

04: Number of bytes

00 00: 50.0rpm Low 2 bytes in memory

42 48:50.0rpm high 2 bytes in memory

33 36: CRC16 low high byte

http://www.styb.cn/cms/ieee_754.php // Single precision byte conversion tool

Set the running time to 10000ms

Command: 01 10 40 05 00 02 04 27 10 00 00 09 22

Command note:

01: Modbus address

10: Modbus function code

30 05: Modbus start address

00 02: Number of registers

04: Number of bytes

27 10:10000ms low 2 bytes in memory

00 00:10000ms high 2 bytes in memory

09 22: CRC16 low high byte

Input register

Address	Parameter	Remarks
0x3001	Firmware version 3+5+8 (major version+minor version+revision version)	
0x3002	Hardware version 3+5+8 (main version+sub version+reserved 00)	
0x3003	Temperature (original value * 10)+500	65535 sensor not connected
0x3004	Float level 0 no alarm 1 alarm	65535 sensor not connected
0x3005	Digital output value, the definition of the value refers to the definition in the rotary encoder	

CRC16 reference code

```
def crc_calculation(buf,lenl):
    i=0
    crc_date=0xffff
    while i<lenl:
        crc_date=crc_date^buf[i]
        j=0
        while j<8 :
            if crc_date&0x0001:
                crc_date=crc_date>>1
                crc_date=crc_date^0xa001
            else:
                crc_date=crc_date>>1
            j+=1
        i+=1
return crc_date
```